

Parallelization of Particle Interpolation and Mover Pipeline

HPC Assignment Report

Parth Karena - 202301472
Pranav Bavadiya - 202301458

April 19, 2026

Contents

1	Parallelization Objective	2
2	Implementation Overview	3
2.1	Pipeline Structure	3
2.2	Parallel Design	3
2.2.1	Pseudocode	4
2.3	Core OpenMP Strategy	4
3	Analysis and Questions	5
3.1	1. Pseudocode of the Parallel Implementation	5
3.2	2. Parallel Implementation Approach and Race Conditions	5
3.3	3. Performance Graphs	5
3.3.1	Lab PCs	6
3.3.2	Cluster	7
3.4	4. Parallel Efficiency and Scalability	9
3.5	5. Comparison with the Serial Version and Maximum Speedup	9
3.6	6. Explanation of the Achieved Speedup	9
3.7	7. Further Optimization with Unlimited HPC Resources	9
3.8	8. Load Imbalance and Performance Bottlenecks	10
3.9	9. Alternative Approach to Improve Performance	10
3.10	10. Other Data Structures or OpenMP-Based Approaches	10
3.11	11. Interpolation vs Mover: Bottleneck Analysis	11
4	Results Summary	12
4.1	Observation Summary	12
4.2	Conclusion	12

Chapter 1

Parallelization Objective

The goal of this assignment is to transform the given serial implementation into an efficient parallel version of the complete pipeline, including interpolation, normalization, mover, denormalization, and void counting. The implementation is organized as a set of separate utility functions, which makes the parallel design clear and modular. The core parallel structure uses a thread-private grid for interpolation and OpenMP-based parallel loops for the remaining stages.

Chapter 2

Implementation Overview

2.1 Pipeline Structure

The program is divided into the following stages:

- **Interpolation:** particles contribute values to nearby grid points.
- **Normalization:** grid values are scaled into a fixed range.
- **Mover:** particle positions are updated using interpolated grid values.
- **Denormalization:** grid values are restored to their original scale.
- **Void Counting:** inactive particles are counted.
- **Saving:** the final mesh is written to output.

2.2 Parallel Design

The implementation avoids race conditions in interpolation by using private grids for each thread and then merging them into the global grid. Normalization and denormalization are parallelized using OpenMP loops, while the mover stage processes particles independently. Void counting uses a reduction-based approach.

2.2.1 Pseudocode

Algorithm 1 Parallel Particle Pipeline

```
1: Initialize global mesh and particle arrays
2: Create thread-private grids for interpolation
3: for all active particles in parallel do
4:   Compute surrounding grid cell
5:   Compute bilinear weights
6:   Add contributions to the local grid of the current thread
7: end for
8: Merge all local grids into the global mesh in parallel
9: Find minimum and maximum mesh values in parallel
10: Normalize mesh values in parallel
11: for all active particles in parallel do
12:   Interpolate grid value at particle position
13:   Update particle coordinates
14:   Mark particle as void if it leaves the domain
15: end for
16: Denormalize mesh values in parallel
17: Count void particles using reduction
18: Save final mesh to file
```

2.3 Core OpenMP Strategy

- **Interpolation:** thread-private accumulation to eliminate atomic updates.
- **Normalization:** parallel min/max reduction followed by parallel rescaling.
- **Mover:** independent particle updates with parallel for scheduling.
- **Denormalization:** parallel inverse scaling of mesh values.
- **Void counting:** reduction over particle states.

Chapter 3

Analysis and Questions

3.1 1. Pseudocode of the Parallel Implementation

The pseudocode is given in the previous section. The corresponding diagram should show the data flow from particles to interpolation, then to normalization, mover, denormalization, and final output.

3.2 2. Parallel Implementation Approach and Race Conditions

The main race condition appears in the interpolation stage because multiple particles may contribute to the same grid points. This is handled by creating one private grid per thread, accumulating contributions locally, and merging the local grids at the end. This removes the need for atomic operations during interpolation and improves scalability.

The mover stage does not require synchronization because each particle is updated independently. Normalization uses parallel reduction for minimum and maximum values, and void counting uses a reduction on the number of inactive particles.

3.3 3. Performance Graphs

The following figures are placed directly in the report.

3.3.1 Lab PCs

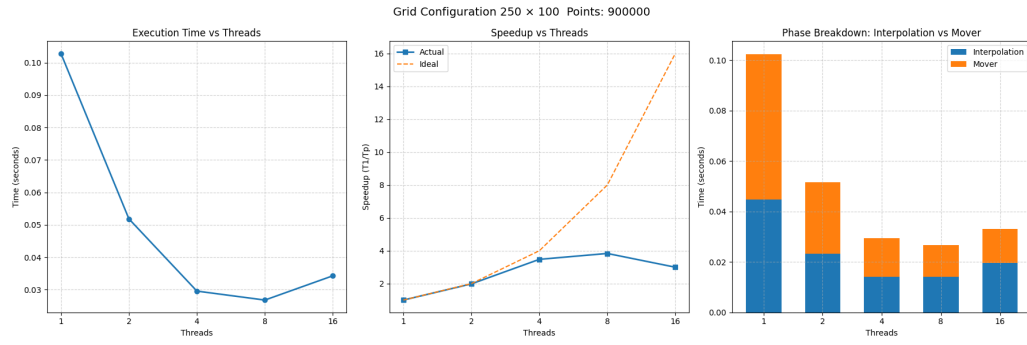


Figure 3.1: Lab PC Graph 1

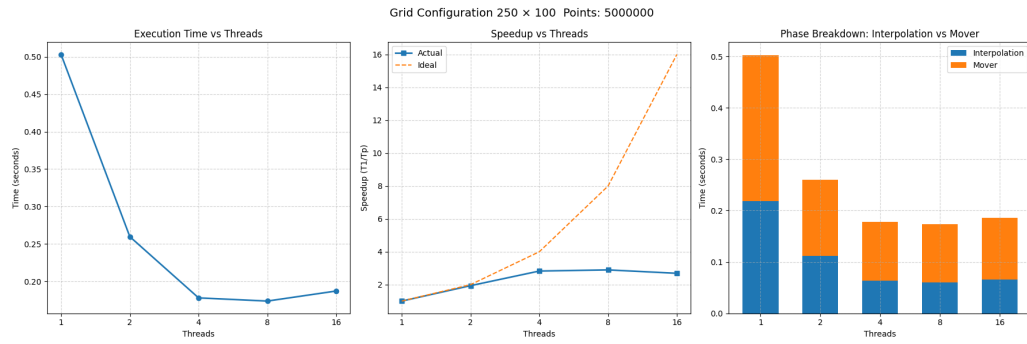


Figure 3.2: Lab PC Graph 2

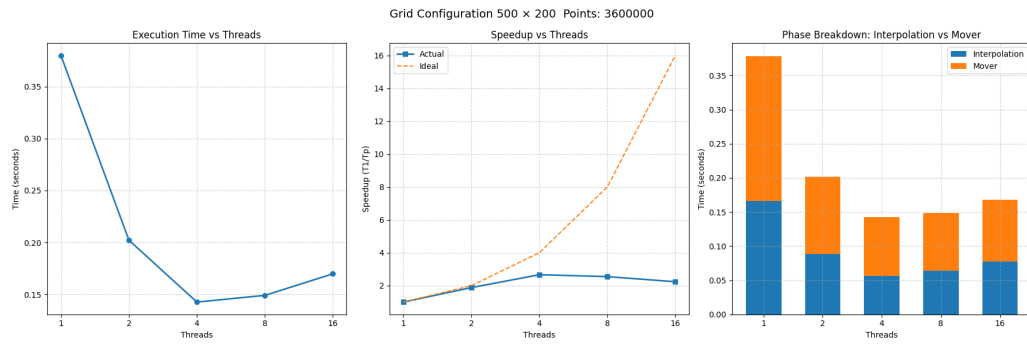


Figure 3.3: Lab PC Graph 3

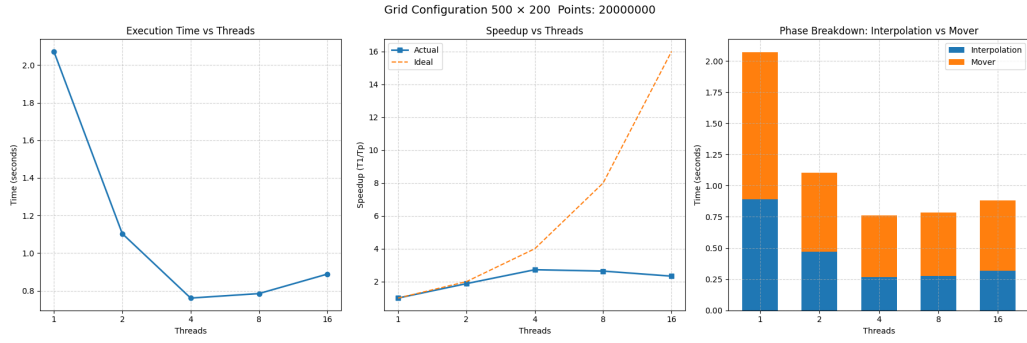


Figure 3.4: Lab PC Graph 4

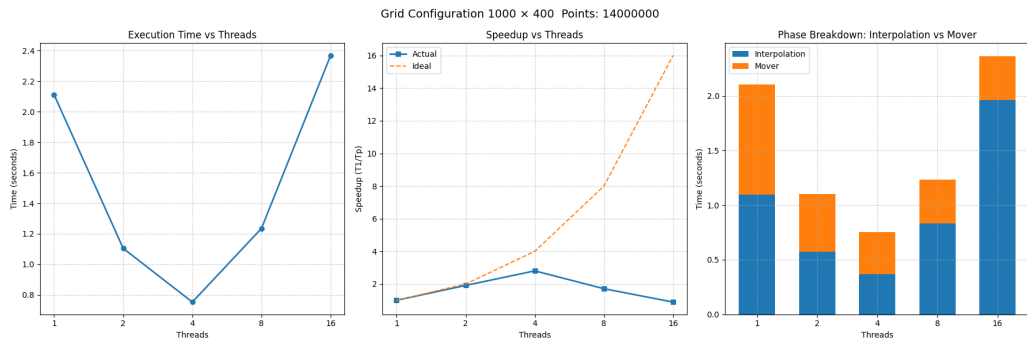


Figure 3.5: Lab PC Graph 5

3.3.2 Cluster

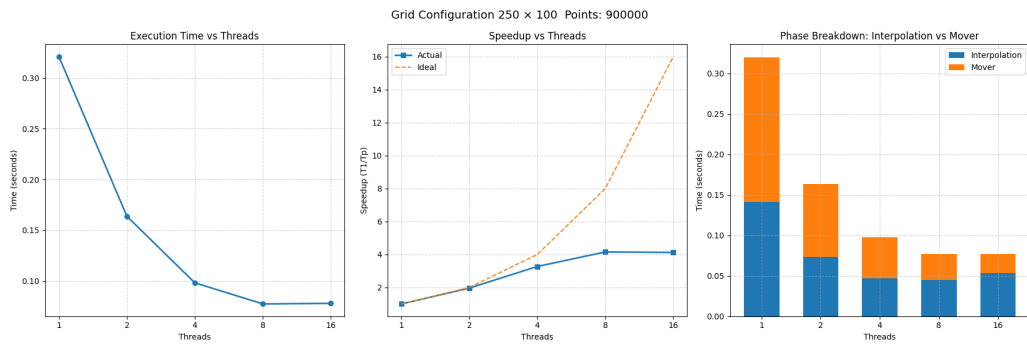


Figure 3.6: Cluster Graph 1

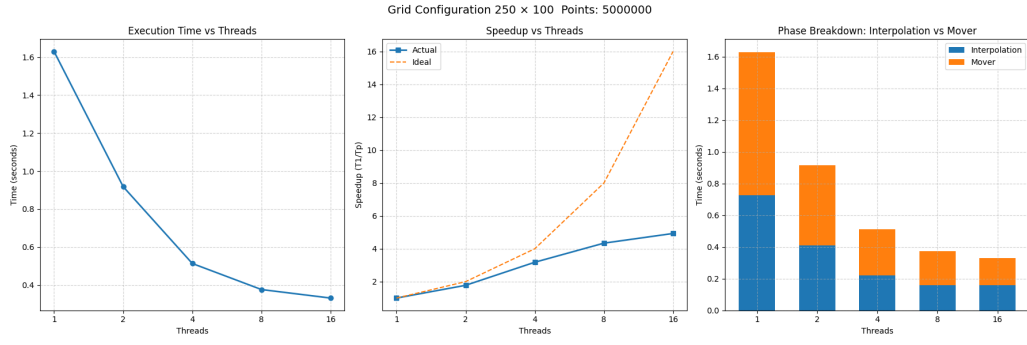


Figure 3.7: Cluster Graph 2

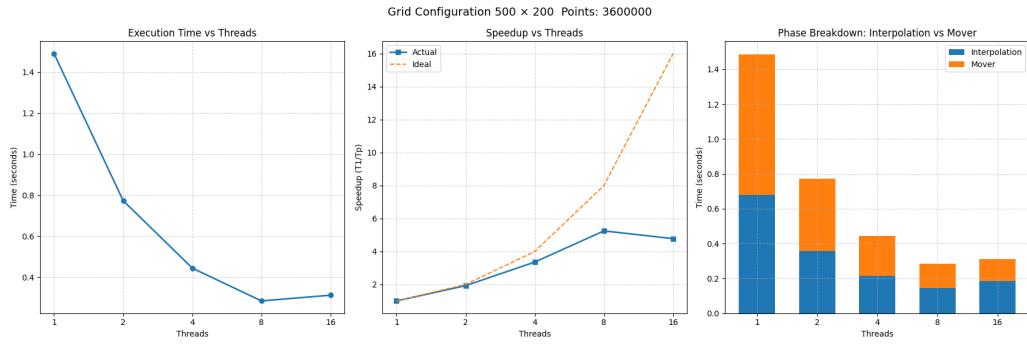


Figure 3.8: Cluster Graph 3

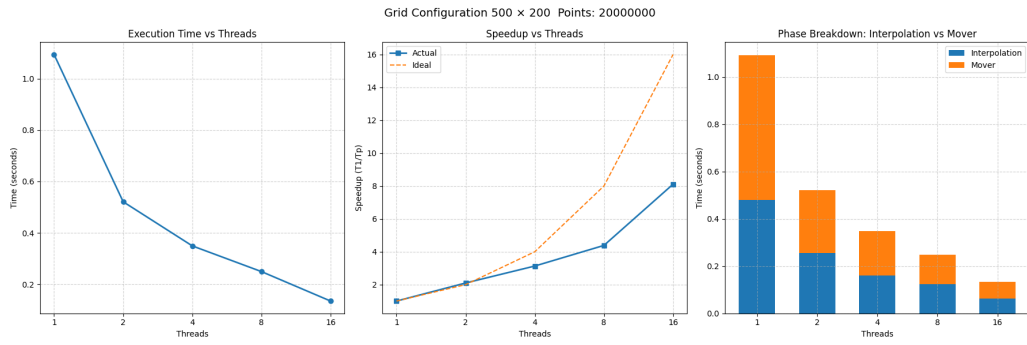


Figure 3.9: Cluster Graph 4

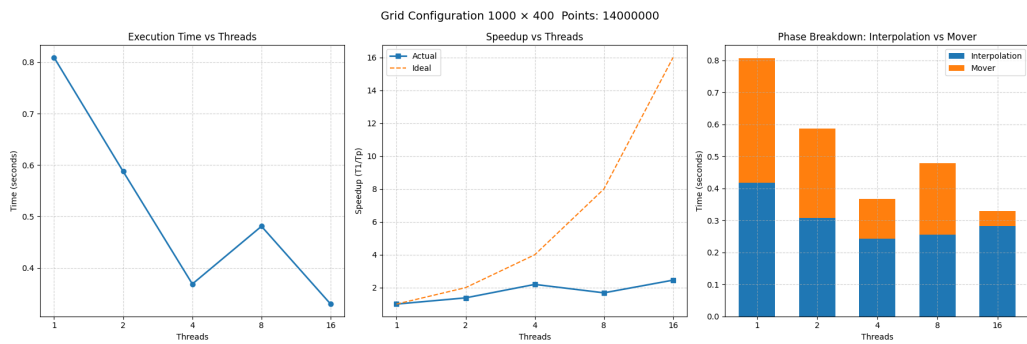


Figure 3.10: Cluster Graph 5

3.4 4. Parallel Efficiency and Scalability

Parallel efficiency can be computed as:

$$E_p = \frac{S_p}{p}$$

where S_p is the speedup on p threads.

The implementation is expected to show strong efficiency at low-to-medium thread counts. When the number of threads increases, the speedup may saturate because of memory bandwidth pressure, synchronization overhead, and cache effects.

3.5 5. Comparison with the Serial Version and Maximum Speedup

The serial execution time is taken from the run with one thread. For each configuration, the maximum speedup is obtained by:

$$S_p = \frac{T_1}{T_p}$$

where T_1 is the time for one thread and T_p is the time for p threads.

Write here: summarize the best speedup observed for each configuration and mention which one achieved the largest overall speedup.

3.6 6. Explanation of the Achieved Speedup

The speedup comes from dividing the work across multiple threads. The largest improvement usually comes from the interpolation and mover stages because they dominate the runtime. The observed scaling is limited by:

- memory access overhead,
- thread creation and synchronization costs,
- load imbalance,
- reduced cache locality at higher thread counts.

Write here: explain the exact trend seen in your graphs.

3.7 7. Further Optimization with Unlimited HPC Resources

If unlimited resources were available, the implementation could be improved by:

- using a better domain decomposition strategy,
- tiling the grid to improve cache locality,
- replacing the dense grid with a sparse or blocked structure when appropriate,
- using SIMD/vectorization for interpolation and mover operations,
- exploring GPU offloading for the particle update phase,
- reducing the cost of the merge step with hierarchical reductions.

3.8 8. Load Imbalance and Performance Bottlenecks

Load imbalance can occur when particles become void and the number of active particles changes over time. This means different threads may receive different amounts of useful work. In addition, interpolation may become a bottleneck because it performs repeated memory updates, and the mover stage can also suffer from poor cache behavior.

Write here: mention which phase was slowest in your measurements and why.

3.9 9. Alternative Approach to Improve Performance

One alternative approach is particle decomposition, where particles are split among threads instead of the grid. Another option is domain decomposition, where the spatial domain is divided into blocks and each thread owns a block. A hybrid method can also be used, combining spatial blocking with privatized accumulation.

3.10 10. Other Data Structures or OpenMP-Based Approaches

Possible alternatives include:

- sparse grid storage,
- blocked 2D arrays for better locality,
- per-thread tile buffers,
- OpenMP tasks for dynamic load balancing,
- atomic updates only for small conflict zones,
- reduction-based merging of subgrids.

Write here: mention which method you would choose and why.

3.11 11. Interpolation vs Mover: Bottleneck Analysis

The interpolation and mover phases should be analyzed separately because they contribute differently to the total runtime. Interpolation usually becomes expensive due to repeated contributions to nearby grid points, while mover is dominated by reading from the grid and updating particle positions.

Write here: compare the phase times from your graphs and explain which phase was the bottleneck for each configuration.

Chapter 4

Results Summary

4.1 Observation Summary

- The parallel pipeline is modular and easy to analyze.
- Privatizing the interpolation grid removes race conditions.
- Normalization and void counting are efficiently parallelized using reductions.
- The mover phase is naturally parallel because each particle is updated independently.
- The final performance depends heavily on grid size, number of particles, and thread count.

4.2 Conclusion

The parallel version preserves correctness while significantly improving runtime over the serial implementation. The best performance is achieved when the workload is large enough to offset thread overhead and when synchronization is minimized.